

Mini Hiring Pipeline

Implementation & Architecture Report

GitHub Repository: <https://github.com/prathamkumar125/Mini-Hiring-Pipeline>

A local recruiting application for tracking candidates through a single hiring pipeline and finding candidates using natural-language search.

1. Project Overview

The Mini Hiring Pipeline is a lightweight recruiter-facing web application. Candidates move through a controlled lifecycle: **Applied** → **Screening** → **Interview** → **Offer** → **Hired**. A candidate may be rejected from an active stage, while Hired and Rejected are final states.

The implementation separates the presentation layer from backend business rules and persistence. It also adds an AI-assisted search layer while keeping database execution deterministic and controlled.

2. Technology Stack

Layer	Technology	Purpose
Frontend	Streamlit	Stage board, candidate form, candidate history, search and logs
Backend	FastAPI	REST API, lifecycle validation and persistence orchestration
Database	SQLite	Candidate records, stage events and search logs
AI / NLP	LangChain + Ollama	Convert natural-language recruiter queries into typed filters
Validation	Pydantic / typed models	Validate search filters and domain data
Testing	pytest	Automated checks

3. Architecture

High-level flow

Streamlit UI → HTTP → FastAPI routes → lifecycle/search services → repository → SQLite

Natural-language search follows a separate controlled path: Search request → LangChain/Ollama → validated SearchFilters → parameterized database query. When AI interpretation is unavailable or unusable, supported deterministic patterns and fuzzy name matching provide a local fallback.

The key architectural decision is that the language model does **not** generate executable SQL. It only produces a small, validated filter representation. Application code turns those filters into parameterized SQL.

4. Core Modules

Module	Responsibility
streamlit_app.py	Recruiter UI: stage board, candidate form, details/history, search results and search logs.
app/main.py	FastAPI application and REST endpoints.
app/lifecycle.py	Validates candidate stage transitions and final-state rules.
app/search.py	Interprets natural-language search requests and selects AI/fallback parsing.
app/repository.py	Reads and writes candidate, event and search-log records.
app/database.py	Creates and manages the SQLite schema and database behavior.
app/domain.py	Defines validated domain/search data models.

5. Candidate Lifecycle Implementation

Candidate movement is intentionally constrained. A candidate can move only to the next stage or to Rejected from an active stage. Hired and Rejected are terminal states. The write is protected by a SQLite transaction so concurrent requests cannot advance the same candidate twice from a stale stage.

Lifecycle: Applied → Screening → Interview → Offer → Hired

Alternative terminal path: Any active stage → Rejected

6. Immutable Audit Trail

Instead of overwriting a candidate's history, each transition is recorded as an append-only stage event. The current stage and time-in-stage are derived from the latest event. SQLite triggers reject edits or deletion of recorded events, preserving the audit trail.

This design allows the recruiter to open a candidate and inspect the complete lifecycle, including how long the candidate has remained in the current stage.

7. Natural-Language Search

The recruiter uses a single search box rather than manually constructing database filters. Example supported queries include:

- **sharam** — fuzzy name matching can find Priya Sharma.
- **Who's in Interview right now?** — filters by current stage.
- **stuck in Screening for more than a week** — combines stage and time-in-stage.
- **moved to Interview since Monday** — searches historical stage events.
- **reached the Offer stage but didn't get hired** — uses lifecycle history.
- **everyone except rejected candidates** — applies a negative stage filter.

8. AI Safety and Deterministic Fallback

LangChain sends the search request to the configured local Ollama model. The model response must validate as a SearchFilters object. The repository then applies those filters using parameterized queries. Model-provided SQL is never executed.

If Ollama is unavailable or returns unusable filters, the application falls back to documented deterministic patterns and fuzzy name matching. The UI communicates when fallback behavior is being used.

9. Search Logging

Each search records the original text, parser source, interpretation, result count, message and timestamp in SQLite. This gives the recruiter visibility into recent AI-assisted searches and makes the search behavior easier to inspect and debug.

10. Installation & Run Instructions

Open PowerShell in the project folder and create the environment:

```
uv venv venv
uv pip install --python venv\Scripts\python.exe -r requirements.txt
```

Create local configuration and install the configured Ollama model:

```
if (-not (Test-Path .env)) { Copy-Item .env.example .env }
ollama pull qwen2.5-coder:1.5b
```

Make sure Ollama is running. Then start FastAPI and Streamlit in separate PowerShell windows:

```
.\venv\Scripts\uvicorn.exe app.main:app --reload
.\venv\Scripts\streamlit.exe run streamlit_app.py
```

Streamlit normally runs at **http://localhost:8501**. FastAPI interactive documentation is available at **http://127.0.0.1:8000/docs**.

Run automated checks with:

```
.\venv\Scripts\python.exe -m pytest
```

11. Configuration

Variable	Default	Purpose
DATABASE_PATH	hiring_pipeline.db	SQLite database location
OLLAMA_MODEL	llama3.2	Local model used to interpret searches; supplied .env.example selects qwen
API_URL	http://127.0.0.1:8000	FastAPI URL used by Streamlit

12. Key Design Decisions

- **Append-only stage events:** preserves an auditable candidate lifecycle.
- **API-enforced transitions:** prevents invalid stage changes and makes rules consistent for UI and direct API clients.
- **Validated filters instead of generated SQL:** limits the authority of the language model and keeps database execution predictable.
- **Local fallback and search logs:** preserves useful search behavior when the local model is unavailable and records how searches were interpreted.
- **SQLite for v1:** keeps deployment lightweight for a single-recruiter local application.

13. Future Improvements

- Add authentication and role-based access.
- Support multiple jobs and pipelines.
- Add database migrations and deployment configuration.
- Expand relative-date and fallback query coverage.
- Add configurable retention for search logs.

- Add dedicated immutable audit events for candidate profile edits.

14. Repository

<https://github.com/prathamkumar125/Mini-Hiring-Pipeline>

Source basis: project README in the public GitHub repository, reviewed for this implementation report.